

Welcome back

How was lab? Was anything confusing?

In []:

Review

We covered:

- What lists are
- List and string indexing
- Modifying lists
- List and string slicing
- Dictionaries (what they are, modification)

In []: `x = [1, True, "hello"]`
`x`

In []: `x = [1, 2.0, True, "hello", [1, 2]]`
`x`

In []: `x[0]`

In []: `x[4]`

In []: `x[-3]`

In []: `x[5]`

In []: `x(0)`

In []: `x`

In []: `x[1] = False`
`x`

In []: `x`

In []: `# list_name[start:end]`
`# IGNORE end`
`x[1:2]`

```
In [ ]: x[1:1]
```

```
In [ ]: x[2:4] # [x[2], x[3]]
```

```
In [ ]: len(x)
```

```
In [ ]: type(x)
```

```
In [ ]: print(x)
```

```
In [ ]: x
```

```
In [ ]: # DO NOT USE x[1:] -> x[1:len(x)]  
# DO NOT USE x[:2] -> x[0:2]  
y = x[1:len(x)]  
y
```

```
In [ ]: x
```

```
In [ ]: # list_name[start:end:step]  
y = x[1:4:2] # [x[1], x[3]]  
y
```

```
In [ ]: x[1:4:1] # [x[1], x[2], x[3]]
```

```
In [ ]: # DO NOT USE x[:3:]  
x[:3:]  
x[0:3]
```

```
In [ ]: x
```

```
In [ ]: y = 1  
z = 3  
x[y:z]
```

```
In [ ]: x[y + 1:z]
```

```
In [ ]: x
```

```
In [ ]: x[-3:-1]
```

```
In [ ]: x[-1:-3]
```

```
In [ ]: x[0:2:-1]
```

```
In [ ]: x
```

```
In [ ]: x[2:0:-1] # [x[2], x[1]]
```

```
In [ ]: x[-1:-3:-1] # [x[-1], x[-2]]
```

```
In [ ]: x[4:2]
```

```
In [ ]: len(x)
```

```
In [ ]: y = "hello"  
y
```

```
In [ ]: y = "hello !"  
len(y)
```

```
In [ ]: y[1:3] # y[1] y[2]
```

```
In [ ]: y[1] = "a"
```

```
In [ ]: address_book = {  
    'Mika': 9174831,  
    'Daniel': 9147382,  
}  
address_book['Daniel']
```

```
In [ ]: x = {  
    [1, 2]: 1,  
}
```

```
In [ ]: x = {  
    1: 2,  
    3: 4,  
    4: 5,  
    6: 7,  
    "hi": "bye",  
    8: [1, 2],  
}  
x
```

```
In [ ]: x[3]
```

```
In [ ]: x[10]
```

```
In [ ]: x["hi"]
```

```
In [ ]: x[1] = 4  
x
```

```
In [ ]: x[1]
```

```
In [ ]: x[10] = 10  
x
```

```
In [ ]: x = {  
        'banana': 1,  
        'apple': 2,  
        }  
  
x['banana'] + x['apple']
```

```
In [ ]: x = [1, 2, 3]  
  
# 1 + 3  
x[0] + x[2]
```

```
In [ ]: x = {  
        'banana': 1,  
        'apple': 2,  
        }  
  
x['banana'] = 3  
x['banana']
```

```
In [ ]: x = {  
        'banana': 1,  
        'apple': 2,  
        }  
  
del x['banana']  
x
```

```
In [ ]: x = {  
        'banana': 1,  
        'apple': 2,  
        }  
  
del x['banana']  
x['banana'] = 3  
x
```

```
In [ ]: x = {  
        0: 1,  
        1: "hello",  
        2: True,  
        }  
  
x = [1, "hello", True]  
  
x[2]
```

Advanced list concepts

Lists can contain any Python "thing", including other lists!

```
In [ ]: x = [1, 2, 3]
x
```

```
In [ ]: y = [x, x]
y
```

```
In [ ]: dictionary = {
    1: 2,
    3: [4, 5],
}

x = [dictionary, 1, "hi"]
x
```

Primitive types vs non-primitive types

`str`, `int`, `float`, and `bool` are primitive types

```
In [ ]: 1.0
```

`list` and `dict` are non-primitive types

```
In [ ]: [1, 2]
```

Passing by *value* vs passing by *reference*

This is a confusing topic. Basic "things" in Python (`int`, `float`, `str`, `bool`) are passed by *value*, which means they are copied when they are assigned to a variable.

```
In [ ]: x = 1
y = x
x = 2

# y
print(x) # x changes
print(y) # y stays as is
```

Lists (and dictionaries) are passed by reference

Lists and dictionaries are not copied. Instead, variables assigned to a list or dictionary store a *reference* to the list/dictionary. Let's see this through examples

```
In [ ]: x = [1, 2, 3]
        y = x
        print(x)
        print(y)
```

```
In [ ]: x = [1, 2, 3]
        y = x
        print(x)
        print(y)
        print()

        x[0] = 5
        print(x)
        print(y)
```

```
In [ ]: x = [1, 2, 3]
        y = x
        print(x)
        print(y)
        print()

        x = [4, 5, 6]
        print(x)
        print(y)
```

```
In [ ]: x = [1, 2, 3]
        y = x
        print(x)
        print(y)
        print()

        y = [4, 5, 6]
        x[0] = 5
        print(x)
        print(y)
```

```
In [ ]: x = [1, 2, 3]
        y = [1, 2, 3]
        print(x)
        print(y)
        print()

        x[0] = 5
        print(x)
        print(y)
```

```
In [ ]: x = [1, 2, 3]
        y = x
        z = y
        print(x)
        print(y)
        print(z)
        print()

        x[0] = 5
        print(x)
        print(y)
        print(z)
```

Review: conditionals

The general syntax for conditionals is

```
if <condition>:
    <expression>
elif <condition>:
    <expression>
else:
    <expression>
```

A <condition> is an <expression that evaluates to a boolean> .

```
In [ ]: numeric_grade = 75

        if numeric_grade > 90:
            letter_grade = 'A'
        elif numeric_grade > 80:
            letter_grade = 'B'
        elif numeric_grade > 70:
            letter_grade = 'C'
        elif numeric_grade > 60:
            letter_grade = 'D'
        else:
            letter_grade = 'F'

        letter_grade
```

Repeated computation

Now that we know what a conditional is, we will use boolean expressions in a different way.

Often times, we want to repeat a computation a number of times.

```
In [ ]: x = [32587, 18, 43, 985, 274398]
        print(x[0])
        print(x[1])
        print(x[2])
        print(x[3])
        print(x[4])
```

```
In [ ]: x = [32587, 18, 43, 985, 274398, 10]
        print(x[0])
        print(x[1])
        print(x[2])
        print(x[3])
        print(x[4])
        print(x[5])
```

We can do this with while loops!

We'll start with some examples before defining the syntax

```
In [ ]: x = [32587, 18, 43, 985, 274398]

        index = 0
        while index < len(x): # len(x) = 5
            print(x[index])
            index += 1

        print("Finished while loop")
```

While loop syntax

The syntax for while loops is:

```
while <condition>:
    <expression>
```

As with `if` statements, the `<condition>` can be anything that evaluates to a boolean. The `<expression>` can be complex.

```
In [ ]: num = 0
        while num < 3:
            print(num)
            num += 1

        print("Finished while loop!")

        # num is 3 at this point
        while num < 3:
            print(num)
            num += 1

        print("Finished the second while loop!")
```

IMPORTANT: THE WHILE LOOP MUST TERMINATE

```
In [ ]: # WARNING
        # WARNING
        # WARNING
        # DO NOT RUN THIS

        while True:
            print('hi')
```

```
In [ ]: # WARNING
        # WARNING
        # WARNING
        # DO NOT RUN THIS

        num = 0
        while num < 10:
            print(num)
```

While loops can contain conditionals!

```
In [ ]: num = 0
        while num < 5:
            if num % 2 == 0:
                print(num)
            num += 1
```

```
In [ ]: num = 0
        if num % 2 == 0:
            while num < 3:
                print(num)
            num += 1
```

You can do complex things now!

```
In [1]: num = 1 # 1
        num = num * 2 # 2
        num = num * 2 # 4
        num = num * 2 # 8
        num
```

Out[1]: 8

```
In [3]: num = 1
        while num < 20:
            print(num)

            num = num * 2
```

```
1
2
4
8
16
```

```
In [33]: sum_of_nums = 10
         # sum_of_nums += 1
         sum_of_nums += 2
         sum_of_nums += 3
         sum_of_nums += 4
         sum_of_nums += 5
         sum_of_nums
```

Out[33]: 24

```
In [35]: # Sum numbers from 1 to n
```

```
sum_of_nums = 0
n = 1000
index = 1
```

```
# Sum to 5
```

```
sum_of_nums += 1
sum_of_nums += 2
sum_of_nums += 3
sum_of_nums += 4
sum_of_nums += 5
sum_of_nums
```

```
sum_of_nums = 0
n = 5
index = 1
```

```
while index <= n:
    sum_of_nums += index
    index += 1
sum_of_nums
```

Out[35]: 24

In [37]: new_variable

```
-----  
NameError                                Traceback (most recent call last)  
Cell In[37], line 1  
----> 1 new_variable  
  
NameError: name 'new_variable' is not defined
```

In [39]:

```
a = 1  
b = 1  
num = 0  
  
while num < 15:  
    print(a)  
    c = a + b  
    a = b  
    b = c  
    num += 1
```

```
1  
1  
2  
3  
5  
8  
13  
21  
34  
55  
89  
144  
233  
377  
610
```

Advanced topic: motivation for functions

Many times, we want to do the same computation on many different inputs.

Right now, it's quite annoying

```
In [41]: n = 5

sum_of_nums = 0
index = 1

# index = 1
# sum_of_nums = 1

# index = 2
# sum_of_nums = 3

# index = 3
# sum_of_nums = 6

# index = 4
# sum_of_nums = 10

# index = 5
# sum_of_nums = 15
while index <= n:
    sum_of_nums += index
    index += 1
sum_of_nums
```

Out[41]: 15

```
In [ ]: x = [[1, 2], [3, 4]]
x[0][0] = x[0][0] * -1
x[0]
x
```

```
In [ ]: n = 4728

sum_of_nums = 0
index = 1

while index <= n:
    sum_of_nums += index
    index += 1
sum_of_nums
```

```
In [43]: # Compute the 8th fibonacci number

a = 1
b = 1
num = 0

while num < 8:
    print(a)
    c = a + b
    a = b
    b = c
    num += 1
```

```
1
1
2
3
5
8
13
21
```

In [45]: *# Compute the 15th fibonacci number*

```
a = 1
b = 1
num = 0

while num < 15:
    print(a)
    c = a + b
    a = b
    b = c
    num += 1
```

```
1
1
2
3
5
8
13
21
34
55
89
144
233
377
610
```

Functions

Can we avoid repeating this kind of computation? Yes we can!

We will use *function*.

As before, we'll start with examples, describe the syntax, and go through more examples.

```
In [51]: def sum_numbers_to_n(n):  
        sum_of_nums = 0  
        index = 1  
  
        while index <= n:  
            sum_of_nums += index  
            index += 1  
        return sum_of_nums  
  
sum_numbers_to_n(10)
```

Out[51]: 55

```
In [90]: def fibonacci(nth):  
        # nth = 8  
        a = 1  
        b = 1  
        num = 0  
  
        while num < nth - 1:  
            c = a + b  
            a = b  
            b = c  
            num += 1  
        return a  
  
fibonacci(8)
```

Out[90]: 21

```
In [55]: fibonacci(15)
```

Out[55]: 610

Syntax for functions

The general syntax for functions is:

```
def function_name(<arguments>):  
    <body>  
    return <expression>
```

There are several things to note about the syntax (we will go through examples):

- You can have zero, one, or more arguments.
- You need the `:` after the function
- The body can be complex!
- You technically don't need a return statement at the end, but we will almost always have a return at the end

```
In [60]: def return_one():
         return 1

         return_one()
```

Out[60]: 1

```
In [88]: def add_two_numbers(x, y):
         # x = 2
         # y = 3
         return x + y

         add_two_numbers(2, 3)
```

Out[88]: 50

```
In [64]: def add_two_numbers(x, y)
         return x + y

         add_two_numbers(10, 20)
```

```
Cell In[64], line 1
      def add_two_numbers(x, y)
      ^
SyntaxError: expected ':'
```

```
In [86]: def add_two_numbers(x, y):
         x + y

         add_two_numbers(10, 20)
```

```
In [68]: # You also need the indentation

         def add_two_numbers(x, y):
         x + y
```

```
Cell In[68], line 4
      x + y
      ^
IndentationError: expected an indented block after function definition on line 3
```

```
In [70]: add_two_numbers(20, 20)
```

Functions are a complex topic!

Pay close attention to the exercises. Think of interesting functions to write!

```
In [ ]: def fahrenheit_to_celsius(f):
         # ???
         return 0
```

```
In [78]: def celsius_to_fahrenheit(c):  
        # ???  
        return 0
```

```
In [80]: celsius_to_fahrenheit(10)
```

```
Out[80]: 0
```

```
In [74]: type(1)
```

```
Out[74]: int
```

```
In [76]: print(1)
```

```
1
```

```
In [92]: x = [1, 2, 3]  
        y = x  
        x[0] = 5  
        x
```

```
Out[92]: [5, 2, 3]
```

```
In [100... x = {  
          1: 2,  
          3: 4,  
          }  
          y = x  
          x[1] = 5  
          y
```

```
Out[100]: {1: 5, 3: 4}
```

```
In [ ]:
```